Ecole Marocaine des Sciences de l'Ingénieur de Tanger

1ère Année Cycle d'Ingénieur

Filière : Ingénierie Informatique et Réseaux

Rapport de Stage

Entreprise d'accueil : TEACHIUM

Développement d'un système de tutorat intelligent basé sur l'IA pour la personnalisation des parcours d'apprentissage

---

Réalisé par : Mohamed Ben Yahya

Organisme d'accueil : TEACHIUM

Localisation : Tétouan, Tanger-Tétouan-Al Hoceïma, Maroc

Durée du stage : 1 mois

Tuteur pédagogique : À compléter

Tuteur dans l'entreprise : À compléter

Année universitaire : 2025–2026

Juillet 2026

Remerciements

Je tiens à exprimer mes sincères remerciements à l'entreprise TEACHIUM pour m'avoir accueilli dans le cadre de ce stage et pour m'avoir donné l'opportunité de travailler sur un sujet actuel combinant e-learning, intelligence artificielle et développement logiciel.

Je remercie également mon encadrant professionnel pour son accompagnement, ses orientations et ses remarques qui m'ont permis de mieux comprendre les besoins réels d'une plateforme d'apprentissage numérique. Mes remerciements s'adressent aussi à mon encadrant pédagogique pour son suivi et ses conseils méthodologiques tout au long de cette expérience.

Enfin, je remercie toutes les personnes qui ont contribué directement ou indirectement à la réalisation de ce travail.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage d'un mois au sein de l'entreprise TEACHIUM, spécialisée dans le développement de contenus e-learning et de solutions d'apprentissage numérique. Le projet porte sur la conception et le développement d'un système de tutorat intelligent basé sur l'intelligence artificielle afin de personnaliser les parcours d'apprentissage des apprenants.

La solution proposée repose sur l'analyse des données d'apprentissage, la modélisation des profils apprenants à l'aide de techniques de machine learning, la génération de recommandations pédagogiques personnalisées, l'intégration d'un assistant conversationnel simple et l'exposition des fonctionnalités via une API REST. Un tableau de bord a également été développé afin de visualiser les indicateurs pédagogiques et les performances des modèles.

Le prototype réalisé utilise un dataset simulé, un algorithme K-Means pour le regroupement des apprenants, un arbre de décision pour la classification des profils, un moteur de recommandation basé sur des règles pédagogiques et une interface de suivi développée avec Streamlit. Les résultats obtenus montrent la faisabilité d'une première version fonctionnelle adaptée à la durée limitée du stage.

Mots clés : e-learning, intelligence artificielle, tutorat intelligent, machine learning, recommandation, API REST, dashboard.

Abstract

This report presents the work carried out during a one-month internship at TEACHIUM, a company specialized in e-learning content development and digital learning solutions. The project focuses on designing and developing an AI-based intelligent tutoring system for personalizing learning paths.

The proposed solution is based on learning data analysis, learner profile modeling using machine learning techniques, personalized pedagogical recommendation, a simple conversational assistant, and REST API integration. A dashboard was also developed to visualize learning indicators and model performance.

The prototype uses a simulated dataset, K-Means clustering for learner grouping, a decision tree model for profile classification, a rule-based recommendation engine, and a Streamlit dashboard. The obtained results demonstrate the feasibility of a first functional version adapted to the limited internship duration.

Key words : e-learning, artificial intelligence, intelligent tutoring system, machine learning, recommendation, REST API, dashboard.

Table des matières

Remerciements **I**

Résumé **II**

Abstract **III**

Liste des abréviations **IX**

Introduction générale **1**

1 Entreprise d'accueil et mission **2**

1.1 Présentation de l'entreprise 2

1.2 Domaine d'activité 2

1.3 Mission confiée 3

1.4 Rôle dans l'équipe 3

2 Cahier des charges et déroulement du stage **4**

2.1 Problématique 4

2.2 Objectifs du projet 4

2.3 Besoins fonctionnels 4

2.4 Besoins non fonctionnels 5

2.5 Déroulement du stage 5

2.6 Diagramme de Gantt 6

2.7 Méthodologie de gestion 6

3 Étude bibliographique **7**

3.1 Systèmes de tutorat intelligent 7
3.2 Machine learning pour le profilage 7
3.3 Moteur de recommandation pédagogique 7
3.4 NLP, LLM et assistance conversationnelle 8
3.5 API REST et dashboard 8

4 Vue conceptuelle 9

4.1 Architecture générale du système 9
4.2 Composants du système 9
4.3 Diagramme de cas d'utilisation 10
4.4 Diagramme de classes 11
4.5 Dictionnaire des données 11
4.6 Choix des modèles IA et justification 12
4.7 Architecture réseau 12
4.8 Modèle BI 12
4.9 Synthèse de la conception 13

5 Implémentation et résultats 14

5.1 Environnement software et hardware 14
5.2 Structure du projet 14
5.3 Préparation des données 15
5.4 Clustering des apprenants 17
5.5 Classification par arbre de décision 17
5.6 Résultats d'évaluation 18
5.7 Moteur de recommandation 19
5.8 API REST 20
5.9 Assistant conversationnel 20
5.10 Dashboard 20
5.11 Tests réalisés 21
5.12 Discussion des résultats avec SWOT 21

Conclusion générale et perspectives **22**

Références bibliographiques **23**

A Commandes d’exécution du projet **25**

B Exemple de requête API **26**

C Limites techniques **27**

Table des figures

4.1 Architecture générale du système de tutorat intelligent 9
4.2 Diagramme de cas d'utilisation du système 10
4.3 Diagramme de classes simplifié du prototype 11

5.1 Distribution des scores moyens des apprenants 15
5.2 Répartition des profils apprenants simulés 16
5.3 Relation entre le temps passé et le score moyen 16
5.4 Méthode Elbow pour le choix du nombre de clusters 17
5.5 Arbre de décision entraîné pour la classification des profils 18
5.6 Matrice de confusion du modèle de classification 19

Liste des tableaux

1.1 Fiche synthétique de TEACHIUM 2

2.1 Besoins fonctionnels du système 5
2.2 Besoins non fonctionnels du système 5
2.3 Planning synthétique du stage 6
2.4 Diagramme de Gantt simplifié 6

4.1 Composants principaux du système 10
4.2 Dictionnaire des données du dataset apprenant 11
4.3 Indicateurs BI proposés 13

5.1 Environnement technique du prototype 14
5.2 Métriques globales du modèle de classification 18
5.3 Exemples de recommandations pédagogiques 19
5.4 Endpoints principaux de l'API 20
5.5 Résumé des tests fonctionnels 21
5.6 Analyse SWOT du prototype 21

Liste des abréviations

Abréviation	**Signification**
API	Application Programming Interface
BI	Business Intelligence
IA	Intelligence Artificielle
LLM	Large Language Model
LMS	Learning Management System
ML	Machine Learning
MVP	Minimum Viable Product
NLP	Natural Language Processing
REST	Representational State Transfer
UML	Unified Modeling Language

Introduction générale

L'évolution rapide des technologies numériques a profondément transformé le domaine de la formation. Les plateformes e-learning permettent aujourd'hui aux apprenants d'accéder à des contenus pédagogiques à distance, de suivre des modules de formation, de passer des quiz et de consulter leur progression. Toutefois, plusieurs plateformes proposent encore des parcours relativement uniformes, sans adaptation suffisante au niveau, au rythme et aux difficultés de chaque apprenant.

Dans ce contexte, l'intelligence artificielle représente une opportunité importante pour améliorer la personnalisation de l'apprentissage. En analysant les données générées par les apprenants, il devient possible d'identifier des profils pédagogiques, de détecter les besoins individuels, de recommander des contenus adaptés et d'accompagner l'utilisateur à travers une assistance conversationnelle.

Le présent stage, réalisé au sein de TEACHIUM, porte sur le développement d'un système de tutorat intelligent basé sur l'IA. Le projet vise à proposer un prototype fonctionnel capable d'analyser les profils apprenants, de générer des recommandations personnalisées, de fournir un assistant conversationnel simple et d'intégrer ces fonctionnalités dans une architecture applicative à travers une API REST.

La problématique principale peut être formulée comme suit : comment concevoir un système intelligent capable d'exploiter les données d'apprentissage pour personnaliser les parcours pédagogiques des utilisateurs d'une plateforme e-learning ?

Les objectifs du projet sont donc multiples. Il s'agit d'abord de préparer des données exploitables, ensuite de modéliser les profils apprenants à l'aide du machine learning, puis de concevoir un moteur de recommandation et un assistant intelligent. Enfin, il faut intégrer ces modules dans un système applicatif simple, testable et présentable.

Ce rapport est structuré en cinq chapitres. Le premier chapitre présente l'entreprise d'accueil et la mission de stage. Le deuxième chapitre détaille le cahier des charges et le déroulement du stage. Le troisième chapitre expose l'étude bibliographique et les notions nécessaires. Le quatrième chapitre présente la conception du système. Le cinquième chapitre décrit l'implémentation, les tests et les résultats obtenus.

1. Entreprise d'accueil et mission

1.1 Présentation de l'entreprise

TEACHIUM est une entreprise spécialisée dans le domaine de l'apprentissage numérique et du développement de contenus e-learning. Elle accompagne les organisations dans la conception de solutions de formation digitales et dans l'utilisation de la technologie pour améliorer les performances d'apprentissage.

L'entreprise est localisée à Tétouan, dans la région Tanger-Tétouan-Al Hoceïma. Elle opère dans le secteur des programmes d'éducation et de formation, avec des spécialités liées au développement de contenus e-learning, aux solutions de formation, à l'expertise en apprentissage numérique et aux systèmes de gestion de l'apprentissage.

Table 1.1 – Fiche synthétique de TEACHIUM

Élément	**Description**
Nom de l'entreprise	TEACHIUM
Secteur	Éducation, e-learning et solutions de formation numérique
Localisation	Tétouan, Tanger-Tétouan-Al Hoceïma, Maroc
Année de création	2010
Taille	Petite structure, environ 2 à 10 employés
Spécialités	E-learning content development, e-learning expertise, learning solutions, LMS
Site web	https://www.teachium.com

1.2 Domaine d'activité

Le domaine d'activité de TEACHIUM se situe à l'intersection de la formation, de la technologie et de l'ingénierie pédagogique. L'entreprise propose des solutions destinées à faciliter la création, la gestion et la diffusion de contenus d'apprentissage numériques.

Dans ce type d'environnement, les données issues des interactions des apprenants peuvent

avoir une grande valeur. Les scores, le temps passé, le taux de complétion, la fréquence de connexion et les feedbacks permettent de mieux comprendre les comportements d'apprentissage et d'améliorer la pertinence des parcours proposés.

1.3 Mission confiée

La mission confiée dans le cadre de ce stage est le développement d'un système de tutorat intelligent basé sur l'intelligence artificielle. Cette mission s'inscrit dans les besoins actuels des plateformes e-learning : accompagner les apprenants de manière personnalisée et fournir aux équipes pédagogiques des indicateurs de suivi pertinents.

La mission est organisée autour de quatre axes principaux :

1. analyse et modélisation des profils apprenants ;
2. conception d'un moteur de recommandation et d'une assistance conversationnelle ;
3. intégration via une API REST et création d'un tableau de bord ;
4. évaluation et optimisation des modèles IA.

1.4 Rôle dans l'équipe

Dans le cadre du stage, le rôle principal consiste à concevoir et développer un prototype fonctionnel. Les tâches réalisées couvrent l'analyse du besoin, la préparation des données, le développement des modèles IA, la création du backend API, la conception du moteur de recommandation, le développement du dashboard et la préparation des livrables techniques.

2. Cahier des charges et déroulement du stage

2.1 Problématique

Les apprenants n'ont pas tous le même niveau, le même rythme, ni les mêmes difficultés. Une plateforme qui propose le même parcours à tous les utilisateurs ne permet pas toujours d'accompagner efficacement chaque apprenant. Un apprenant en difficulté a besoin de révision et d'exercices guidés, tandis qu'un apprenant avancé a plutôt besoin de contenus plus complexes ou de projets pratiques.

La problématique du projet est donc la suivante : comment concevoir un système intelligent capable d'analyser les données d'un apprenant, d'identifier son profil pédagogique et de lui proposer un accompagnement adapté ?

2.2 Objectifs du projet

L'objectif général du projet est de développer un prototype de système de tutorat intelligent pour personnaliser les parcours d'apprentissage sur une plateforme e-learning.

Les objectifs spécifiques sont les suivants :

— collecter ou simuler des données d'apprentissage exploitables ;
— analyser les comportements des apprenants ;
— regrouper les apprenants par profils à l'aide du clustering ;
— classifier les apprenants selon leur profil pédagogique ;
— générer des recommandations personnalisées ;
— proposer une assistance conversationnelle simple ;
— exposer les fonctionnalités via une API REST ;
— visualiser les indicateurs dans un tableau de bord ;
— évaluer les performances du prototype.

2.3 Besoins fonctionnels

Table 2.1 – Besoins fonctionnels du système

Code	Besoin fonctionnel
BF01	Le système doit recevoir les données d'un apprenant.
BF02	Le système doit analyser les indicateurs pédagogiques de l'apprenant.
BF03	Le système doit identifier un profil pédagogique.
BF04	Le système doit générer des recommandations adaptées au profil.
BF05	Le système doit fournir une réponse conversationnelle simple.
BF06	Le système doit exposer ses fonctionnalités via une API REST.
BF07	Le système doit afficher les statistiques pédagogiques dans un dashboard.
BF08	Le système doit fournir des métriques d'évaluation du modèle.

2.4 Besoins non fonctionnels

Table 2.2 – Besoins non fonctionnels du système

Code	Besoin non fonctionnel
BNF01	Le système doit être simple à utiliser et facile à tester.
BNF02	L'architecture doit être claire et maintenable.
BNF03	Les traitements doivent être suffisamment rapides pour un usage interactif.
BNF04	Le code doit être organisé et documenté.
BNF05	Les résultats doivent être interprétables pour un contexte pédagogique.
BNF06	Le prototype doit pouvoir évoluer vers une intégration avec des données réelles.

2.5 Déroulement du stage

La durée du stage étant limitée à un mois, le travail a été organisé en quatre phases principales. La première semaine est consacrée à l'analyse du besoin et à la préparation des données. La deuxième semaine porte sur les modèles IA et le moteur de recommandation. La troisième semaine concerne l'intégration API REST et le dashboard. La dernière semaine est réservée aux tests, à l'optimisation et à la rédaction.

Table 2.3 – Planning synthétique du stage

Période	**Objectif**	**Livrables**
Semaine 1	Cadrage et données	Cahier des charges, dataset, premières visualisations
Semaine 2	IA et recommandation	Clustering, arbre de décision, moteur de recommandation, chatbot
Semaine 3	Intégration	API REST, endpoints, dashboard
Semaine 4	Validation	Tests, métriques, rapport, présentation

2.6 Diagramme de Gantt

Table 2.4 – Diagramme de Gantt simplifié

Tâche	**S1**	**S2**	**S3**	**S4**
Analyse du besoin	X			
Cahier des charges	X			
Préparation du dataset	X			
Analyse exploratoire	X	X		
Clustering et classification		X		
Moteur de recommandation		X		
Assistant conversationnel		X	X	
API REST			X	
Dashboard			X	
Tests et évaluation				X
Rapport et présentation				X

2.7 Méthodologie de gestion

La méthodologie adoptée est incrémentale. Chaque composant est développé sous forme de module indépendant, puis intégré progressivement dans le système complet. Cette approche est adaptée à un stage court, car elle permet d'obtenir rapidement un prototype démontrable tout en limitant les risques techniques.

3. Étude bibliographique

3.1 Systèmes de tutorat intelligent

Un système de tutorat intelligent est un système informatique destiné à accompagner l'apprenant de manière personnalisée. Contrairement à un système d'apprentissage classique, il cherche à adapter les ressources, les exercices et les explications au profil de l'utilisateur. Dans le cadre de ce projet, l'adaptation repose sur les données d'apprentissage : scores, complétion, activité et feedback.

3.2 Machine learning pour le profilage

Le machine learning permet d'extraire des régularités à partir des données. Dans ce projet, deux familles de méthodes sont utilisées : les méthodes non supervisées et les méthodes supervisées.

Le clustering K-Means est utilisé pour regrouper automatiquement les apprenants selon leurs caractéristiques. La documentation officielle de scikit-learn présente K-Means comme un algorithme de regroupement paramétré notamment par le nombre de clusters et l'initialisation des centroïdes [1]. Cette méthode est pertinente pour une première analyse de profils, car elle ne nécessite pas forcément des étiquettes prédéfinies.

L'arbre de décision est utilisé pour classifier les apprenants selon leur profil pédagogique. Dans scikit-learn, `DecisionTreeClassifier` permet de traiter des problèmes de classification binaire ou multiclasse [2]. Ce modèle a été retenu parce qu'il est interprétable et adapté à une explication pédagogique des décisions.

3.3 Moteur de recommandation pédagogique

Un moteur de recommandation vise à proposer une ressource adaptée à un utilisateur. Dans ce projet, le moteur est volontairement simple et fondé sur des règles pédagogiques. Ce choix est justifié par la durée limitée du stage, le besoin d'interprétabilité et l'absence éventuelle de données réelles en grande quantité.

Par exemple, un apprenant en difficulté reçoit des cours de révision et des quiz de renforcement, tandis qu'un apprenant avancé reçoit des contenus plus complexes et des mini-projets. Ce moteur peut évoluer vers des techniques plus avancées si des historiques réels d'apprentissage deviennent disponibles.

3.4 NLP, LLM et assistance conversationnelle

Le traitement du langage naturel permet à un système de comprendre ou de générer du texte. Dans ce prototype, l'assistance conversationnelle reste simple : elle repose sur une détection de mots-clés et sur des réponses adaptées au profil de l'apprenant. Cette approche permet de démontrer l'utilité d'un assistant pédagogique sans dépendre d'un modèle coûteux.

Dans une version future, l'assistant pourrait être enrichi par une approche de type retrieval-augmented generation. Les travaux de Lewis et al. introduisent des modèles combinant une mémoire paramétrique et une mémoire non paramétrique récupérée par recherche documentaire, afin d'améliorer les tâches de NLP nécessitant des connaissances [5]. Une telle approche permettrait de répondre à partir des supports pédagogiques de TEACHIUM.

3.5 API REST et dashboard

FastAPI a été choisi pour exposer les fonctionnalités du système à travers une API REST. Le framework repose sur les standards ouverts OpenAPI et JSON Schema et fournit une documentation automatique des modèles de données [3]. Cela facilite les tests via Swagger et l'intégration avec une plateforme e-learning.

Streamlit a été choisi pour créer rapidement un dashboard interactif en Python. La documentation officielle indique que Streamlit permet de construire des applications de données avec des graphiques, widgets et mises en page interactives [4]. Ce choix est adapté au prototype, car il réduit le temps nécessaire au développement frontend.

4. Vue conceptuelle

4.1 Architecture générale du système

Le système proposé est organisé autour de quatre couches : données, intelligence artificielle, API REST et interface de visualisation. La couche données contient les informations liées aux apprenants. La couche IA analyse les profils et génère les recommandations. La couche API rend les fonctionnalités accessibles. La couche interface permet de visualiser les statistiques et de tester le prototype.

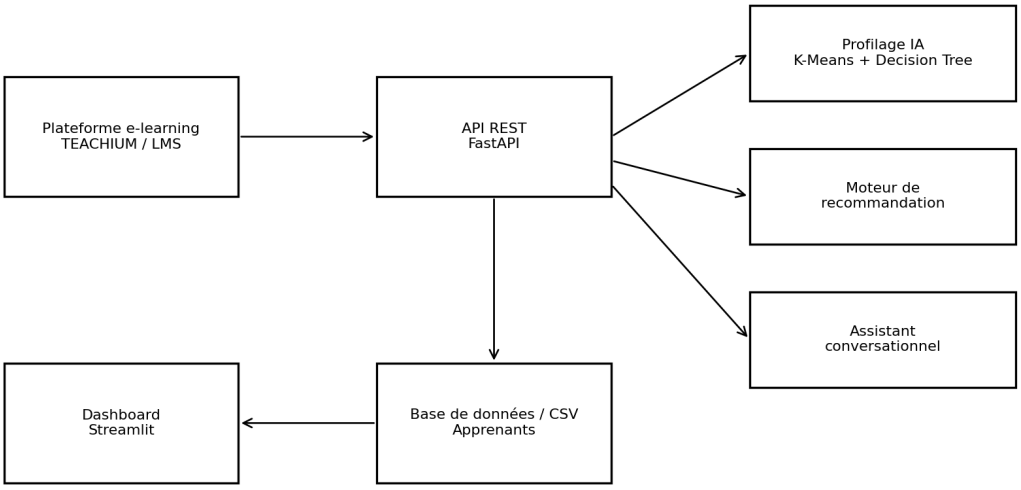


Figure 4.1 – Architecture générale du système de tutorat intelligent

La figure précédente montre le flux principal du système. La plateforme e-learning ou un utilisateur envoie les données à l'API REST. L'API appelle les services IA, notamment le modèle de profilage, le moteur de recommandation et l'assistant conversationnel. Les données et les résultats sont ensuite visualisés dans le dashboard.

4.2 Composants du système

Table 4.1 – Composants principaux du système

Composant	**Rôle**
Dataset apprenant	Contient les indicateurs pédagogiques utilisés par les modèles IA.
Profile Service	Analyse les données et retourne un profil pédagogique.
Recommendation Service	Génère une liste de ressources adaptées au profil.
Chatbot Service	Répond à des questions simples liées au profil ou aux recommandations.
FastAPI Controller	Expose les endpoints REST du système.
Dashboard Service	Affiche les statistiques et permet de tester le prototype.

4.3 Diagramme de cas d'utilisation

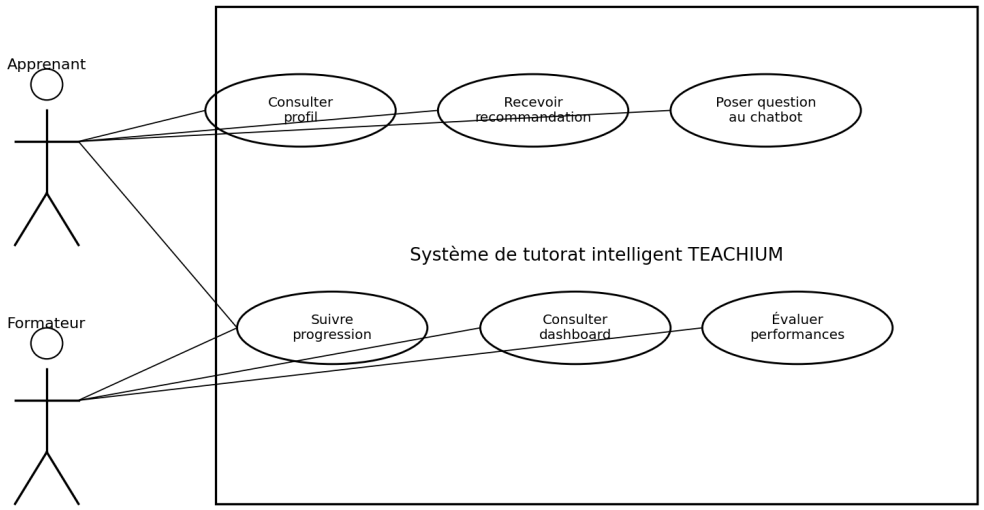


Figure 4.2 – Diagramme de cas d'utilisation du système

Le diagramme de cas d'utilisation montre les interactions principales. L'apprenant peut consulter son profil, recevoir une recommandation et poser une question à l'assistant. Le formateur peut suivre la progression à travers le dashboard. L'administrateur peut consulter les performances globales du système.

4.4 Diagramme de classes

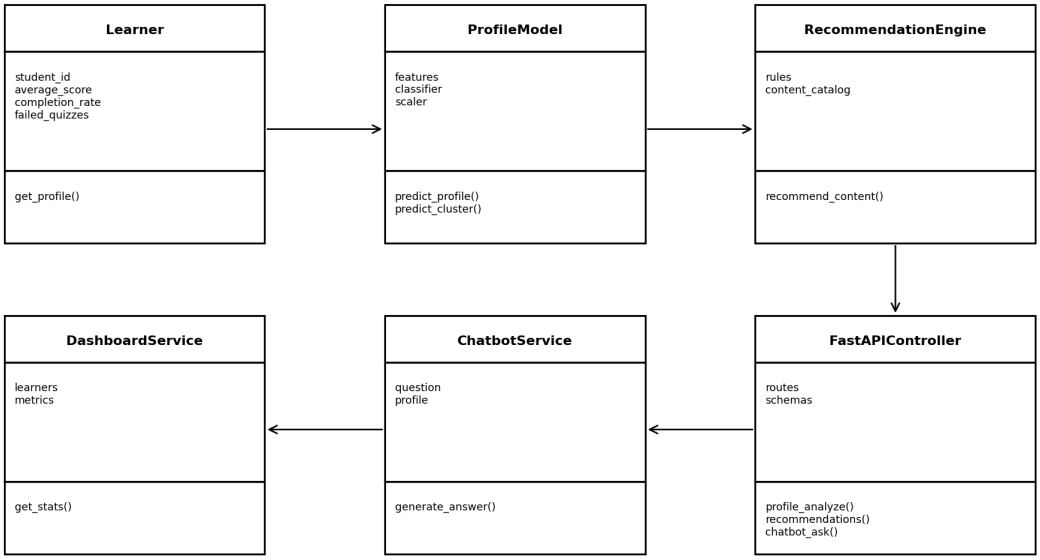


Figure 4.3 – Diagramme de classes simplifié du prototype

Le diagramme de classes présente les principales entités logicielles. La classe `Learner` représente les données de l'apprenant. Le `ProfileModel` effectue la prédiction du profil. Le `RecommendationEngine` génère les recommandations. Le `ChatbotService` fournit les réponses conversationnelles. Le `FastAPIController` expose ces fonctionnalités sous forme d'endpoints.

4.5 Dictionnaire des données

Table 4.2 – Dictionnaire des données du dataset apprenant

Champ	**Type**	**Description**
student_id	Chaîne	Identifiant anonyme de l'apprenant.
course_category	Chaîne	Catégorie principale du contenu suivi.
average_score	Numérique	Score moyen obtenu dans les quiz.
quiz_attempts	Entier	Nombre total de quiz passés.
completed_courses	Entier	Nombre de cours terminés.
total_courses	Entier	Nombre total de cours affectés à l'apprenant.
time_spent_hours	Numérique	Temps passé sur la plateforme en heures.

Champ	Type	Description
completion_rate	Numérique	Pourcentage de complétion du parcours.
login_frequency	Entier	Nombre moyen de connexions par semaine.
failed_quizzes	Entier	Nombre de quiz échoués.
satisfaction_score	Numérique	Note de satisfaction de 1 à 5.
true_profile	Chaîne	Profil pédagogique simulé utilisé comme cible de classification.
cluster	Entier	Groupe obtenu par K-Means.
profile	Chaîne	Interprétation pédagogique du cluster.

4.6 Choix des modèles IA et justification

Le choix de K-Means est justifié par sa simplicité, sa rapidité et sa capacité à regrouper les apprenants sans étiquettes initiales. Il permet de découvrir des profils à partir de plusieurs indicateurs numériques.

Le choix de l'arbre de décision est justifié par son interprétabilité. Dans un contexte pédagogique, il est important de comprendre pourquoi un apprenant est classé comme débutant, moyen, avancé ou en difficulté. Un arbre de décision permet de relier les variables d'entrée aux décisions de manière plus transparente que certains modèles plus complexes.

Le moteur de recommandation est basé sur des règles parce que le stage dure un mois et que les données réelles peuvent être limitées. Cette approche permet de produire rapidement un système cohérent, explicable et évolutif.

4.7 Architecture réseau

L'architecture réseau du prototype est locale. Le backend FastAPI est lancé sur `localhost:8000`. Le dashboard Streamlit est lancé séparément sur un port local. Dans une intégration future, l'API pourrait être déployée sur un serveur de TEACHIUM et consommée directement par la plateforme LMS.

4.8 Modèle BI

Le modèle BI du prototype repose sur quatre familles d'indicateurs : activité, performance, progression et satisfaction. Ces indicateurs permettent d'alimenter le dashboard et d'aider les formateurs à suivre l'état des apprenants.

Table 4.3 – Indicateurs BI proposés

Dimension	**Indicateurs**
Performance	Score moyen, nombre de quiz réussis, nombre de quiz échoués.
Progression	Taux de complétion, nombre de cours terminés.
Activité	Temps passé, fréquence de connexion, nombre de quiz passés.
Satisfaction	Score de satisfaction utilisateur, feedback.

4.9 Synthèse de la conception

La conception proposée répond à une logique modulaire. Chaque composant possède un rôle bien défini : le module de données prépare les variables, le module IA détecte le profil, le moteur de recommandation propose les ressources, l'assistant conversationnel explique les résultats et l'API REST assure la communication entre les parties. Cette séparation facilite la maintenance et permet de remplacer un module sans modifier toute l'application.

Cette architecture est également adaptée à une évolution future. Par exemple, le dataset simulé peut être remplacé par une base de données réelle, le moteur de recommandation par règles peut être enrichi par un modèle supervisé, et le chatbot simple peut évoluer vers un assistant LLM connecté aux contenus pédagogiques de TEACHIUM. Le prototype constitue donc une base technique claire pour une intégration progressive dans un environnement e-learning réel.

Du point de vue pédagogique, le système ne se limite pas à prédire un profil. Il explique la décision et propose une action concrète : réviser, pratiquer, reprendre un parcours court ou passer à un niveau avancé. Cette logique rend le système plus utile pour l'apprenant et plus compréhensible pour l'équipe pédagogique.

5. Implémentation et résultats

5.1 Environnement software et hardware

Le prototype a été développé dans un environnement Python et peut être ouvert dans Visual Studio Code. Les technologies principales utilisées sont Python, FastAPI, scikit-learn, pandas, matplotlib et Streamlit.

Table 5.1 – Environnement technique du prototype

Élément	**Description**
Éditeur	Visual Studio Code
Langage	Python
Backend	FastAPI
Machine learning	scikit-learn
Données	CSV simulé
Visualisation	matplotlib, Streamlit
Tests	pytest et Swagger

5.2 Structure du projet

Listing 5.1 – Structure principale du projet

```
teachium_ai_tutor_project/
 app/
    main.py
    schemas/
    services/
    models/
    data/
 dashboard/
    dashboard.py
 scripts/
    generate_dataset.py
    train_models.py
```

```
figures/
reports/
   rapport_stage_teachium.tex
tests/
requirements.txt
README.md
```

Cette organisation sépare clairement les responsabilités. Le dossier `app` contient le backend et les services métiers. Le dossier `scripts` contient les scripts de génération des données et d'entraînement. Le dossier `dashboard` contient l'interface de suivi. Les figures et le rapport sont séparés afin de faciliter la documentation.

5.3 Préparation des données

En l'absence de données réelles directement exploitables, un dataset simulé a été généré. Ce dataset contient 350 apprenants et plusieurs variables pédagogiques. Les profils simulés sont : avancé, moyen, débutant, en difficulté et peu actif.

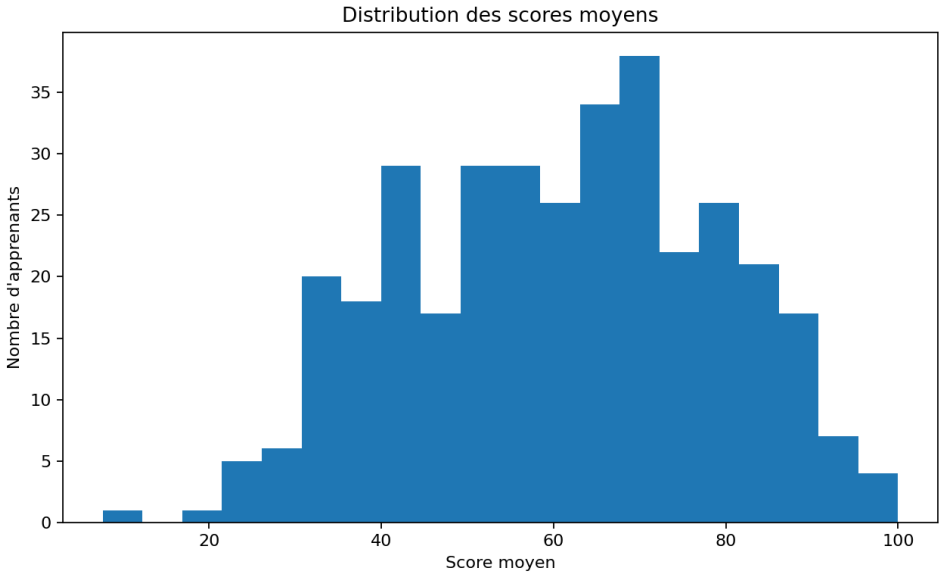


Figure 5.1 – Distribution des scores moyens des apprenants

La figure montre une distribution variée des scores. Cette diversité est importante pour tester la capacité du système à différencier les profils pédagogiques.

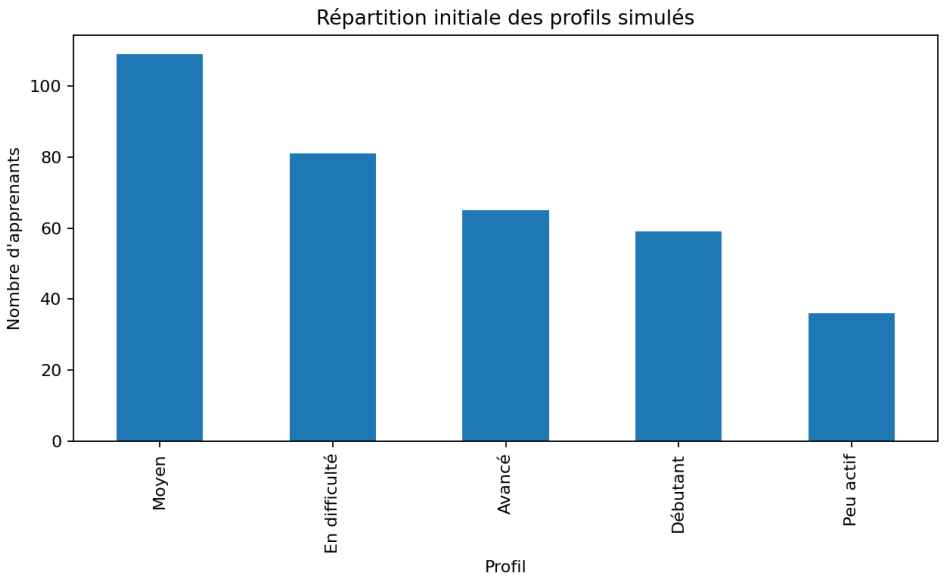


Figure 5.2 – Répartition des profils apprenants simulés

La répartition des profils permet de vérifier que le dataset contient plusieurs catégories d'apprenants. Le système peut ainsi être testé sur des cas différents : apprenants avancés, apprenants moyens, apprenants débutants, apprenants en difficulté et apprenants peu actifs.

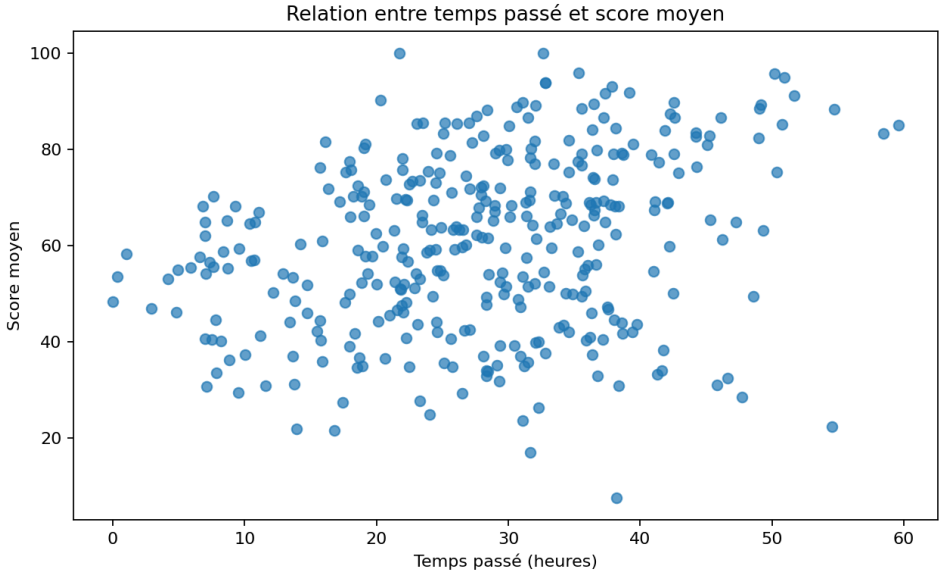


Figure 5.3 – Relation entre le temps passé et le score moyen

Cette visualisation permet d'observer la relation entre l'engagement et la performance. Un temps élevé ne garantit pas toujours un score élevé, ce qui justifie l'utilisation de plusieurs indicateurs pour définir un profil.

5.4 Clustering des apprenants

Le clustering a été réalisé avec K-Means après normalisation des variables numériques. La méthode Elbow a été utilisée pour observer l'évolution de l'inertie selon le nombre de clusters.

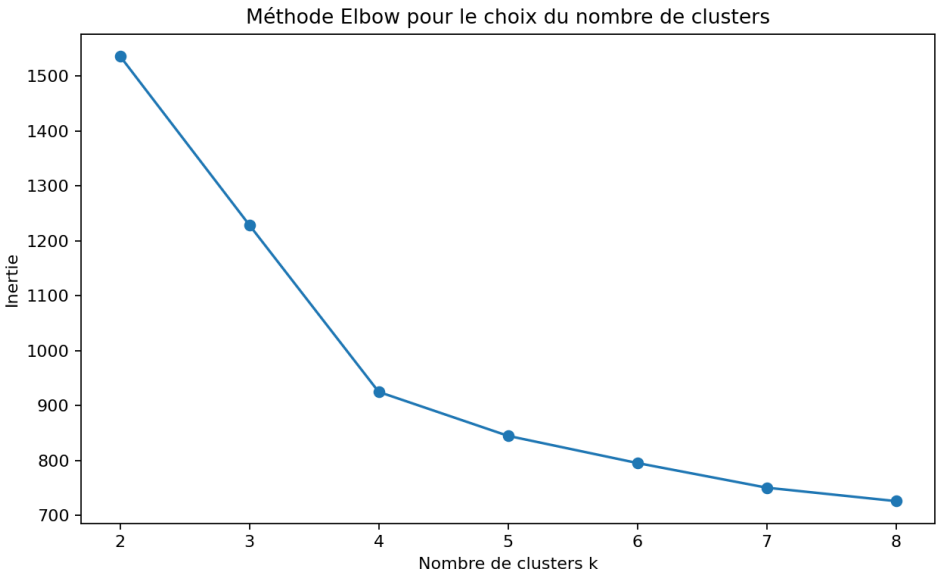


Figure 5.4 – Méthode Elbow pour le choix du nombre de clusters

La figure montre l'évolution de l'inertie lorsque le nombre de clusters augmente. Pour ce prototype, cinq clusters ont été retenus afin de correspondre aux cinq profils pédagogiques principaux.

5.5 Classification par arbre de décision

Un modèle `DecisionTreeClassifier` a été entraîné pour prédire le profil d'un apprenant à partir des variables pédagogiques. Les variables utilisées sont le score moyen, le temps passé, le taux de complétion, le nombre de quiz passés, le nombre de quiz échoués, la fréquence de connexion et la satisfaction.

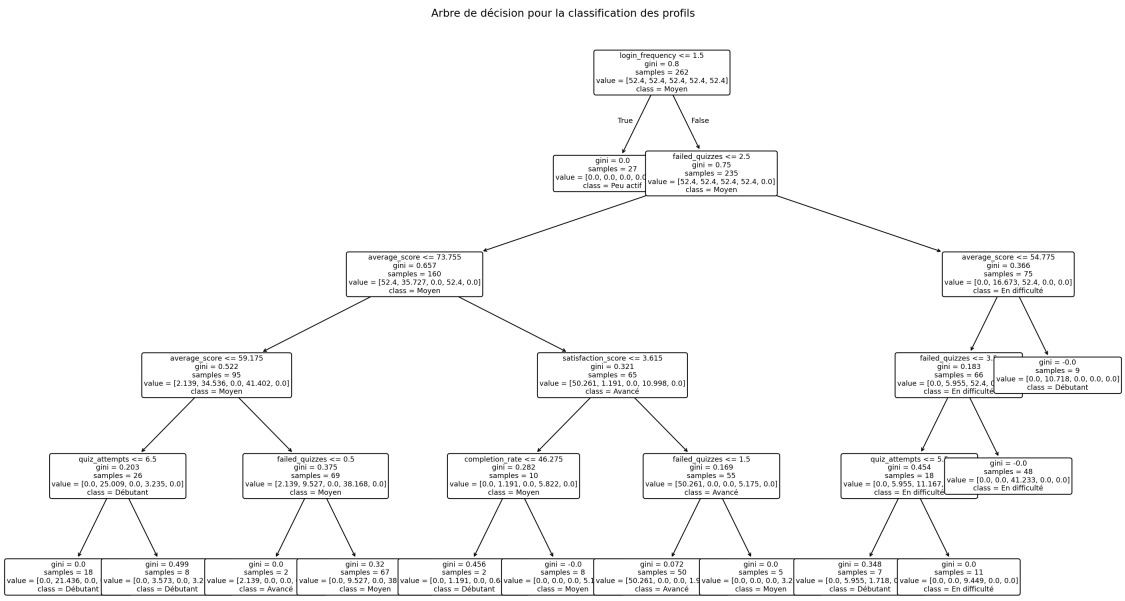


Figure 5.5 – Arbre de décision entraîné pour la classification des profils

L'arbre de décision permet d'interpréter les décisions du modèle. Il montre que les variables comme le score moyen, la complétion et les quiz échoués jouent un rôle important dans la séparation des profils.

5.6 Résultats d'évaluation

Le modèle de classification a été évalué sur un jeu de test représentant 25% du dataset. Les résultats obtenus sont présentés dans le tableau suivant.

Table 5.2 – Métriques globales du modèle de classification

Métrique	**Valeur**
Accuracy	0.8409
Precision pondérée	0.8610
Recall pondéré	0.8409
F1-score pondéré	0.8459

Ces résultats montrent que le modèle parvient à reconnaître correctement la majorité des profils simulés. Les performances sont satisfaisantes pour un prototype, mais elles doivent être interprétées avec prudence car les données sont simulées.

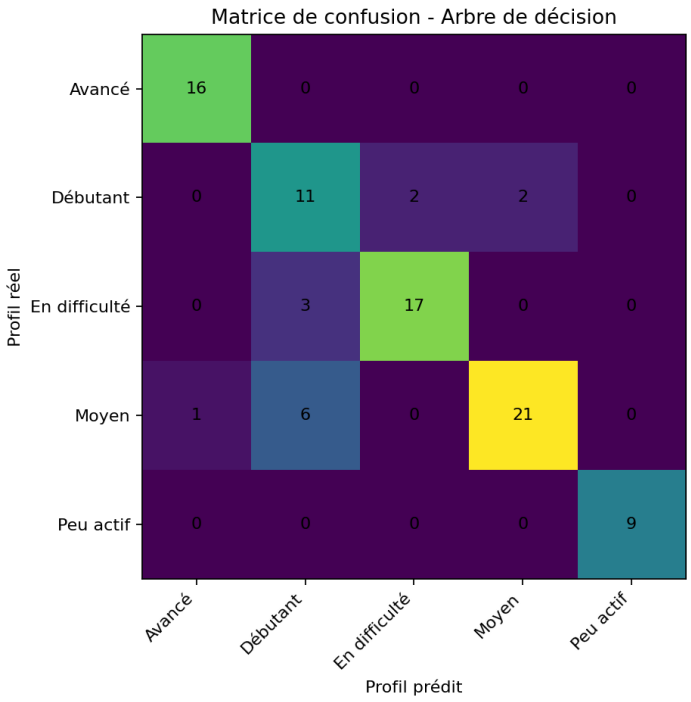


Figure 5.6 – Matrice de confusion du modèle de classification

La matrice de confusion permet d'identifier les confusions entre profils. Les profils les plus distincts, comme *Avancé* et *Peu actif*, sont mieux reconnus. Les profils intermédiaires, comme *Débutant* et *Moyen*, peuvent être plus proches et donc plus difficiles à séparer.

5.7 Moteur de recommandation

Le moteur de recommandation repose sur des règles pédagogiques simples. Ces règles associent chaque profil à un type de contenu adapté.

Table 5.3 – Exemples de recommandations pédagogiques

Profil	**Recommandation**
En difficulté	Cours de révision, vidéo explicative, quiz de renforcement.
Débutant	Module d'introduction, exercices guidés, quiz facile.
Moyen	Exercices standards, quiz de progression, résumé de cours.
Avancé	Contenu avancé, mini-projet pratique, quiz supérieur.
Peu actif	Parcours court, rappel motivationnel, objectif hebdomadaire simple.

Cette approche est simple, mais elle présente un avantage important : elle est explicable. L'équipe pédagogique peut comprendre pourquoi une ressource a été proposée à un apprenant.

5.8 API REST

Les fonctionnalités du système sont exposées à travers une API REST développée avec FastAPI. Les endpoints principaux sont présentés ci-dessous.

Table 5.4 – Endpoints principaux de l'API

Endpoint	Description
`GET /`	Vérifie que l'API fonctionne.
`POST /api/profile/analyze`	Analyse les données d'un apprenant et retourne son profil.
`POST /api/recommendations`	Retourne des recommandations selon le profil.
`POST /api/chatbot/ask`	Répond à une question posée à l'assistant.
`GET /api/dashboard/stats`	Retourne les statistiques du dashboard.
`GET /api/model/evaluate`	Retourne les métriques d'évaluation du modèle.

5.9 Assistant conversationnel

L'assistant conversationnel est une première version fondée sur des mots-clés. Il peut répondre à des questions concernant le profil, les recommandations, les conseils de progression ou les cours à revoir. Cette approche constitue une base simple qui pourra être remplacée par un assistant LLM dans une version future.

Listing 5.2 – Exemple simplifié de réponse conversationnelle

```
def chatbot_response(question, profile, recommended_content=None):
    q = question.lower()
    if "profil" in q:
        return f"Votre profil actuel est : {profile}."
    if "recommand" in q:
        return "Cette ressource est recommandée selon votre profil."
    return "Je peux vous aider à comprendre votre parcours."
```

5.10 Dashboard

Un dashboard Streamlit a été développé pour visualiser les données et tester le prototype. Il affiche le nombre d'apprenants, le score moyen, le taux de complétion moyen, la satisfaction moyenne, la répartition des profils et des graphiques exploratoires.

Le dashboard contient également un formulaire permettant de saisir les données d'un apprenant et d'obtenir automatiquement son profil et ses recommandations. Cela facilite la démonstration du système auprès de l'encadrant.

5.11 Tests réalisés

Les tests ont porté sur les endpoints de l'API, le fonctionnement du moteur de recommandation et l'analyse de profil. Les tests unitaires simples ont été réalisés avec `pytest`.

Table 5.5 – Résumé des tests fonctionnels

Test	**Résultat attendu**	**Statut**
Route racine	Message de fonctionnement de l'API	Validé
Analyse profil	Retour d'un profil pédagogique	Validé
Recommandation	Liste de contenus recommandés	Validé
Chatbot	Réponse textuelle adaptée	Validé
Dashboard	Affichage des indicateurs	Validé

5.12 Discussion des résultats avec SWOT

Table 5.6 – Analyse SWOT du prototype

Élément	**Analyse**
Forces	Architecture claire, prototype fonctionnel, API testable, résultats interprétables, dashboard simple.
Faiblesses	Données simulées, chatbot limité, recommandation basée sur des règles, absence d'intégration réelle avec un LMS.
Opportunités	Intégration avec les données réelles de TEACHIUM, amélioration par LLM, personnalisation plus avancée, suivi pédagogique en temps réel.
Menaces	Qualité insuffisante des données, contraintes de confidentialité, coût d'un LLM, difficulté d'intégration avec l'existant.

Conclusion générale et perspectives

Ce stage a permis de concevoir et développer un prototype de système de tutorat intelligent basé sur l'intelligence artificielle pour la personnalisation des parcours d'apprentissage. Le système réalisé répond aux objectifs principaux : analyser les données d'un apprenant, détecter son profil pédagogique, générer des recommandations adaptées, fournir une assistance conversationnelle simple et afficher les indicateurs dans un dashboard.

Les résultats obtenus montrent qu'une première version fonctionnelle est réalisable dans le cadre d'un stage d'un mois. Le modèle de classification atteint une accuracy de 0.8409 sur les données simulées, ce qui constitue un résultat satisfaisant pour un prototype. Le moteur de recommandation permet de relier chaque profil à des ressources pédagogiques adaptées, tandis que l'API REST rend le système testable et intégrable.

L'impact potentiel pour TEACHIUM est important. Une telle solution peut aider l'entreprise à enrichir ses services e-learning par une personnalisation plus fine, un meilleur suivi des apprenants et une aide intelligente à la décision pédagogique. Pour la formation personnelle du stagiaire, ce projet a permis de mobiliser des compétences en développement backend, machine learning, visualisation de données, conception logicielle et documentation technique.

Cependant, le prototype présente certaines limites. Les données utilisées sont simulées et ne remplacent pas des données réelles. Le chatbot reste simple et ne comprend pas réellement le contenu des cours. Le moteur de recommandation est basé sur des règles et non sur un historique réel d'interactions. Ces limites constituent des axes d'amélioration pour une version future.

Les perspectives possibles sont l'intégration de données réelles anonymisées, l'utilisation d'un modèle LLM connecté aux supports pédagogiques de TEACHIUM, l'amélioration du moteur de recommandation par apprentissage, l'ajout d'un système de feedback utilisateur et le déploiement de l'API sur un serveur de production.

Références bibliographiques

Bibliographie

[1] Scikit-learn, *KMeans - scikit-learn documentation*. Disponible sur : https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html. Consulté en 2026.

[2] Scikit-learn, *Decision Trees - scikit-learn documentation*. Disponible sur : https://scikit-learn.org/stable/modules/tree.html. Consulté en 2026.

[3] FastAPI, *FastAPI documentation - Features*. Disponible sur : https://fastapi.tiangolo.com/features/. Consulté en 2026.

[4] Streamlit, *Get started with Streamlit - Streamlit Docs*. Disponible sur : https://docs.streamlit.io/get-started. Consulté en 2026.

[5] P. Lewis, E. Perez, A. Piktus et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, arXiv :2005.11401, 2020.

[6] TEACHIUM, *Site web officiel*. Disponible sur : https://www.teachium.com. Consulté en 2026.

A. Commandes d'exécution du projet

Listing A.1 – Installation et lancement du projet

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts/generate_dataset.py
python scripts/train_models.py
uvicorn app.main:app --reload
streamlit run dashboard/dashboard.py
```

B. Exemple de requête API

Listing B.1 – Payload JSON pour l'analyse d'un apprenant

```
{
  "average_score": 45,
  "time_spent_hours": 12,
  "completion_rate": 40,
  "quiz_attempts": 5,
  "failed_quizzes": 3,
  "login_frequency": 2,
  "satisfaction_score": 2.5
}
```

C. Limites techniques

Le prototype ne remplace pas une plateforme LMS complète. Il constitue une preuve de concept. Pour une mise en production, il serait nécessaire d'ajouter une authentification, une base de données robuste, une gestion des droits, un système de logs, une stratégie de confidentialité des données et une intégration réelle avec l'environnement technique de TEACHIUM.